

PROGRAMMING IN JAVA / PROJECT REPORT

Campus Lost & Found Matcher

A terminal application for campus reports and matching

STUDENT

Yatharth Singh

COURSE

Programming in Java

REGISTRATION NUMBER

25BAI10657

REPORT DATE

17 September 2026

Structured reports, ranked suggestions, and local CSV storage. The project contains ten application source files, one test source file, and three functional modules.

REPORT CONTENTS

02 Introduction

03 Problem Statement

04 Functional Requirements

05 Non-functional Requirements

06 System Architecture

07 Design Diagrams

08 Design Decisions & Rationale

09 Implementation Details

10 Screenshots / Results

11 Testing Approach

12 Challenges Faced

13 Learnings & Key Takeaways

14 Future Enhancements

15 References

SECTION 02

Introduction

A student loses a bottle in the library. Another student finds it and leaves it at a help desk. Both reports may exist, yet the students still need a way to connect them. Campus Lost & Found Matcher records the details in one place and suggests reports that may describe the same item.

The application runs entirely in a Java terminal. A student or help-desk operator can record an item, search existing reports, review suggested matches, and update the report when the case is resolved. Records remain available after the application closes because they are saved in a local CSV file.

Three modules handle the work: Report Management, Matching Engine, and Search & Reporting. The implementation uses Java classes, inheritance, enums, collections, exception handling, dates, and file input/output. The JDK is sufficient to compile and run both the application and its tests.

The objectives are practical: capture consistent information, reduce the number of reports that must be compared manually, explain each match score, and keep open and resolved cases easy to distinguish.

SECTION 03

Problem Statement

Lost-and-found messages are difficult to manage when they are spread across notice boards and group chats. Descriptions vary, older messages are hard to locate, and resolved cases may still appear active. This project addresses the record-keeping and comparison part of that problem.

The intended users are students and campus help-desk staff sharing one installation. Each report records whether an item was lost or found, along with its title, description, category, location, incident date, contact reference, and current status.

The scope includes adding, viewing, editing, deleting, resolving, and reopening reports. It also includes ranked suggestions, combined search filters, and basic statistics. Accounts, notifications, image matching, and communication between separate computers are outside this version.

The application assumes that incident dates are reasonably accurate and that the computer clock is correct. Its data folder must allow writes and atomic file replacement. A file lock allows only one application instance to work on a particular dataset at a time.

A successful result is a consistent set of records and useful suggestions for follow-up. A high score does not confirm ownership, and the application does not record a verified relationship between a lost report and a found report.

SECTION 04

Functional Requirements

ID / Module	Required behaviour	Evidence
FR1 / Reports	Create LOST or FOUND with unique ID and saved data.	Service + CLI tests
FR2 / Reports	View details; edit while preserving ID and type.	Service + CLI tests
FR3 / Reports	Delete only after explicit confirmation.	CLI cancellation and deletion
FR4 / Reports	Resolve/reopen and persist the status.	Service + CLI tests
FR5 / Matching	Filter, score, threshold, and rank eligible pairs.	Matcher boundary tests
FR6 / Matching	Show point breakdown, day gap, and shared keywords.	Matcher tests + captured output
FR7 / Search	Combine text/category/location/type/status/date filters.	Service + CLI tests
FR8 / Reporting	Count reports by type, status, and category.	Service tests + demo output

Report forms accept single-line text, listed category and status values, and dates in YYYY-MM-DD format. Creating a report returns a unique ID. Searches return matching records; the matcher adds scores and their component values. Statistics display counts of report rows.

The main menu has five actions and an exit option. Longer lists pause after five records. Invalid entries can be corrected at the prompt. A completed action returns to the menu, while the end of input closes the application without saving an unfinished form.

SECTION 05

Non-functional Requirements

Attribute	Acceptance target
Reliability	Reload preserves records. A failed save does not update service memory. A second app cannot obtain the same data lock.
Usability	Five home actions plus exit, five-record pages, clear prompts, and ASCII/no-colour fallback. No user study has been conducted.
Maintainability	Ten production files separate UI, models, services, storage, and validation. Business logic can be tested without keyboard interaction.
Error handling	Reject invalid dates/text/enum values; report malformed CSV with row context and leave input unchanged.
Performance	Rank 10,000 bounded-length synthetic candidates for one source in under 2 seconds on the development laptop. Measure matching, not startup or save time.
Bounded scalability	Handle the synthetic 10,000-report matching case and document whole-file rewrites, in-memory storage, and lack of distributed access.

These targets describe the intended behaviour of this local prototype. Automated tests check data handling and the main workflows. The timing result covers one matching run on the development laptop. Usability has not been measured in a student study, and the project has not been evaluated as a production service.

SECTION 06

System Architecture

01 / System architecture

Single installation. Clear presentation, service, model, and storage boundaries.

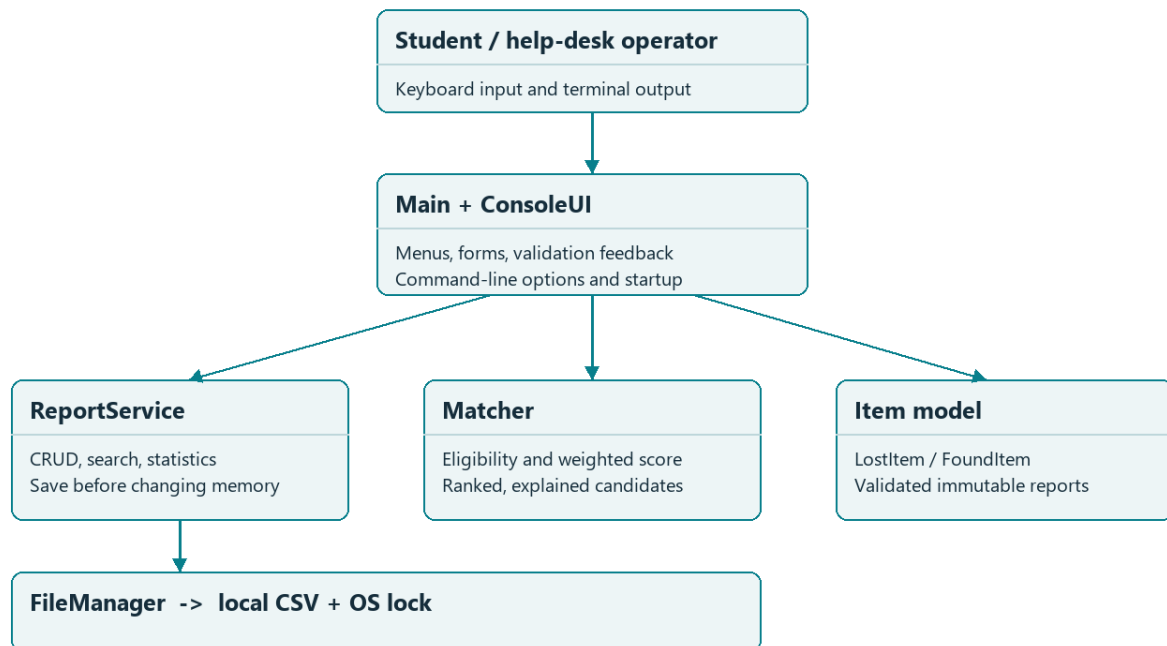


Figure 1. Architecture and local persistence boundary.

Main starts the application, selects the data file, and handles menu actions. ConsoleUI presents prompts and results. ReportService maintains the collection of reports and saves changes through FileManager. Matcher works with report objects to produce ranked suggestions.

Item defines the common report fields. LostItem and FoundItem provide the two report types. ValidationUtil checks inputs, and InvalidInputException represents errors that the user can correct. These responsibilities are separated so that storage and matching can be tested without navigating terminal menus.

SECTION 07

Design Diagrams

03 / Use case diagram

One shared-terminal actor; no login, remote messaging, or automatic ownership claim.

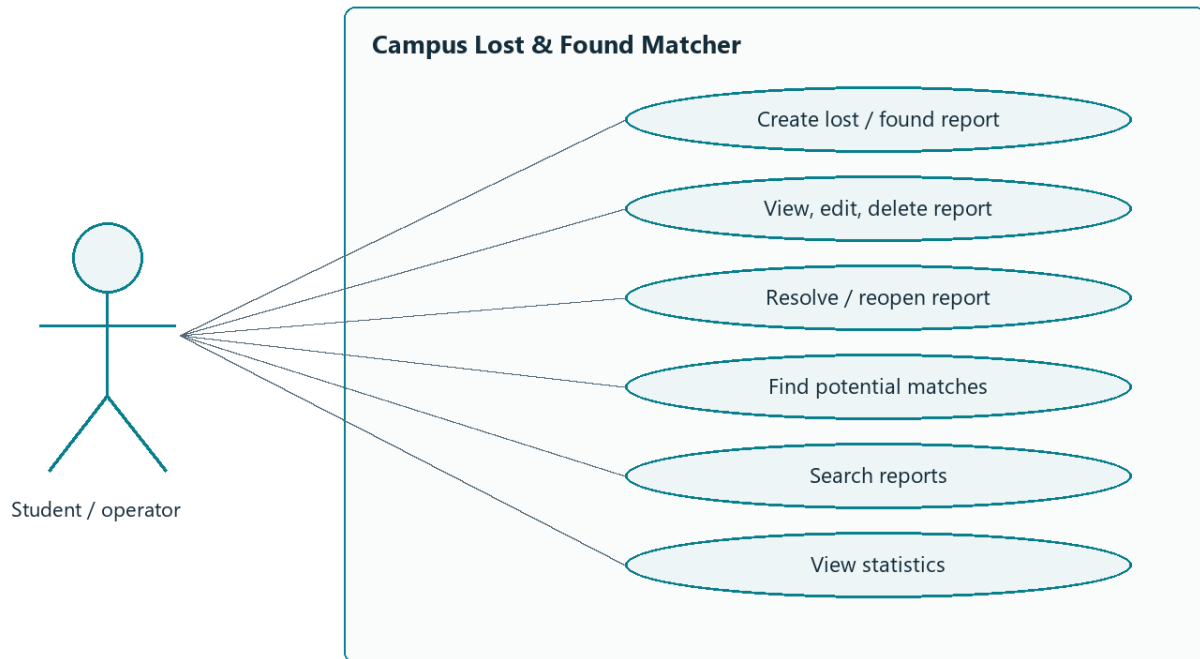
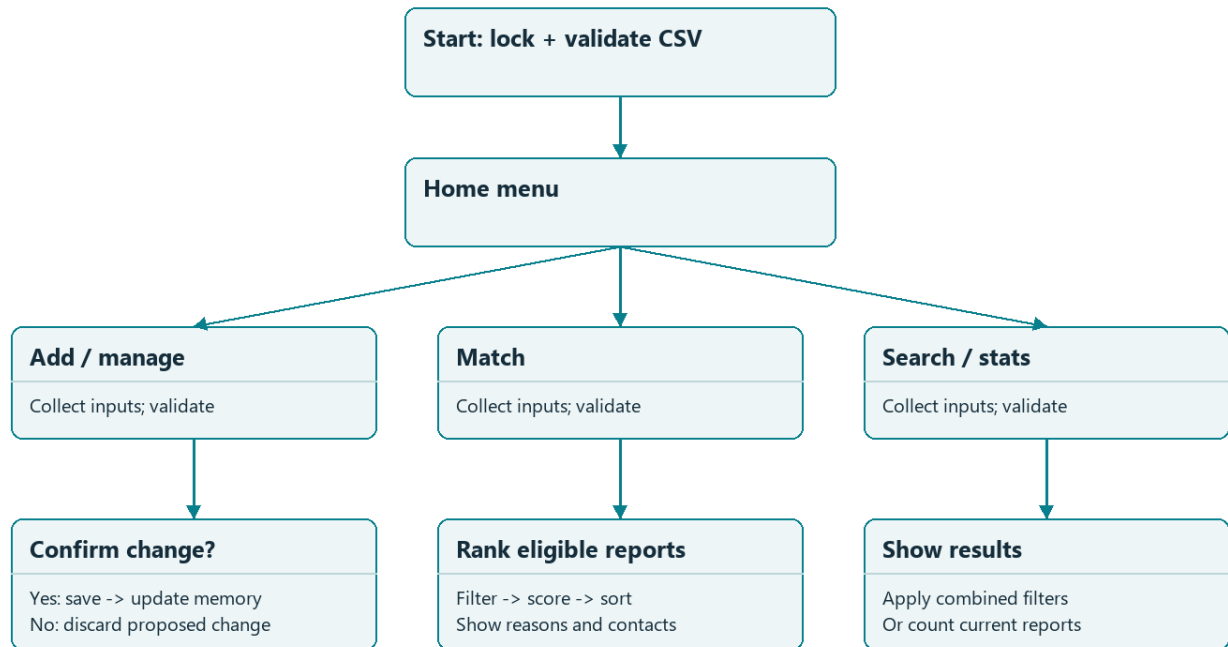


Figure 2. The shared-terminal actor and supported use cases.

Report workflow

02 / Workflow

Every confirmed mutation saves immediately; unresolved forms create no records.



All completed actions return to Home. Invalid fields retry; failed operations show an error.

Home: 0 -> release lock -> exit. EOF -> discard unfinished form -> release lock -> exit.

Startup error -> explain problem -> exit with code 1; do not overwrite damaged input.

Figure 3. Startup, actions, validation, persistence, and exit.

Classes and their responsibilities

04 / Class diagram

Representative members and dependencies. Hollow arrows mean inheritance.

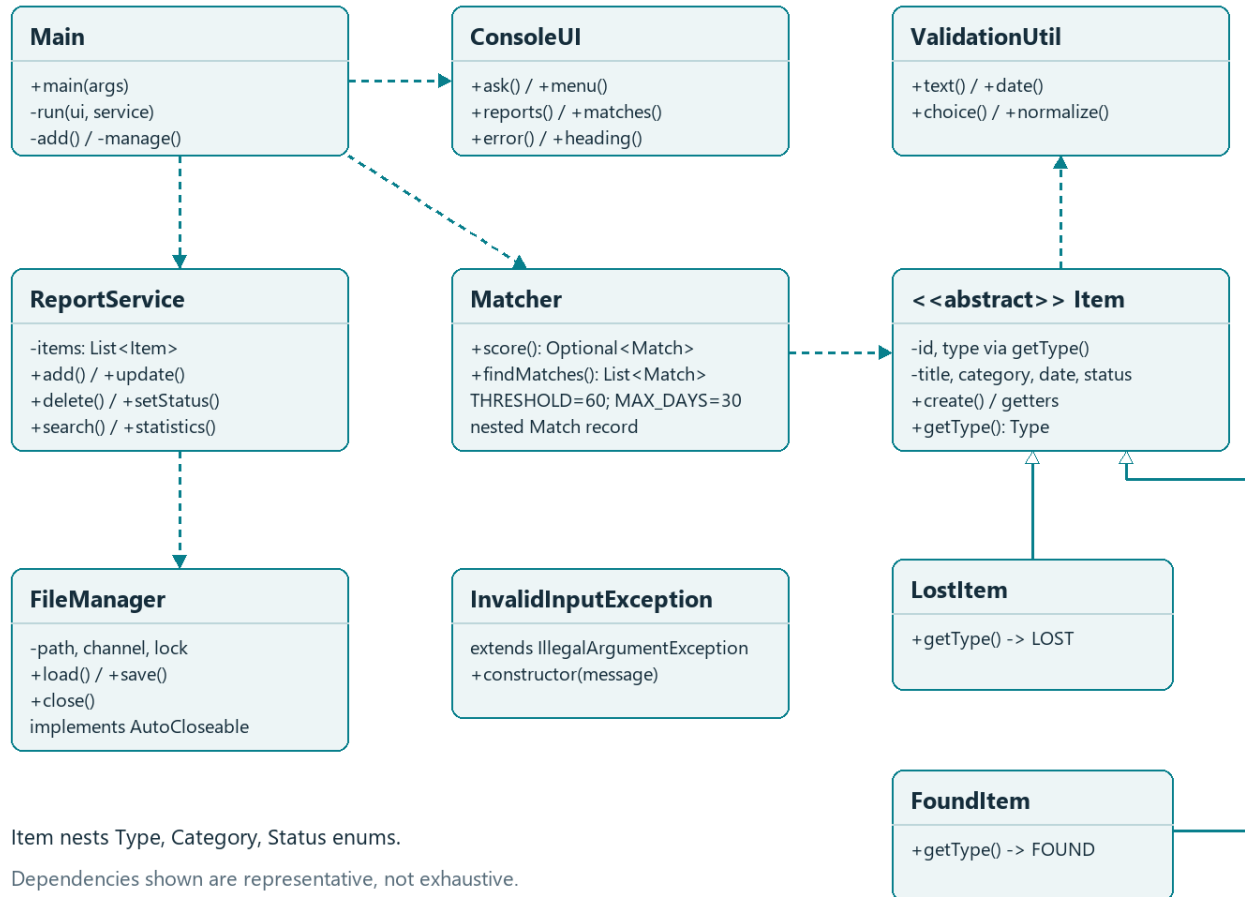


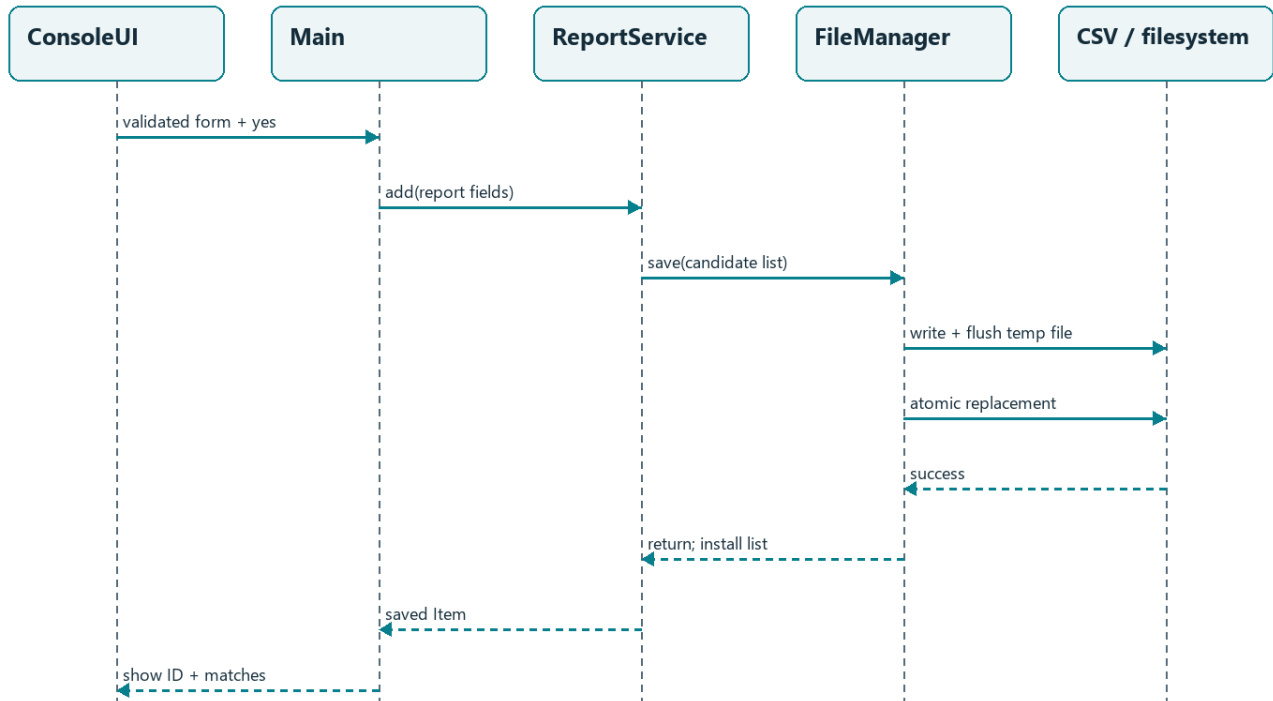
Figure 4. Ten production classes with representative dependencies and inheritance.

The diagram shows the ten application classes and their main relationships. Type, Category, and Status are enums inside Item. Match is a record inside Matcher that stores the candidate and its score breakdown. Dashed arrows show dependencies; hollow arrowheads show inheritance.

Saving a new report

05 / Sequence: add a report

Only a successful disk replacement allows the service to publish new in-memory state.



If write/move fails: IOException returns to Main; show error; keep the previous service list.

Figure 5. The service publishes changed memory only after persistence succeeds.

The save completes before the service replaces its current in-memory list. If writing or replacing the file fails, an IOException is reported and the previous list remains available. Matching can then run after a report has been saved successfully.

Storage schema

06 / ER and storage schema

One stored entity. Matching results are derived and have no persistent relationship.

REPORT | data/items.csv

PK id	string / UUID generated by app
type	LOST FOUND
title	required, <= 80 characters
description	required, <= 500 characters
category	one of seven categories
location	required, <= 80 characters
date	ISO date, not in the future
contact	required, <= 120 characters
status	OPEN RESOLVED

LOST and FOUND share the same CSV schema; type selects the Java subclass.

No user table, match table, foreign keys, or invented relationships are stored.

Figure 6. A single stored Report entity with id as its primary key.

Each CSV row represents one report. Its fields are id, type, title, description, category, location, date, contact, and status. The type field distinguishes LOST from FOUND. Match results are calculated when requested and are not stored as a separate entity.

Titles and locations allow up to 80 characters, descriptions 500, and contact references 120. Incident dates must be valid and cannot be in the future. CSV fields are quoted, with internal quotes doubled. Newlines and control characters are rejected so that each report occupies one row.

SECTION 08

Design Decisions & Rationale

Decision	Reason and trade-off
Java CLI	Satisfies terminal execution and demonstrates course concepts. No GUI setup is needed. Rich visual interactions are outside scope.
Plain javac/java	No build-tool or test-framework dependencies. Argument files list source paths explicitly. The JDK remains a prerequisite.
CSV + memory	Readable storage and simple setup. Every mutation rewrites all reports, and searches scan memory; it is not a multi-user database.
Weighted heuristic	Transparent, deterministic scoring. Weights and threshold are assumptions, not trained or empirically calibrated values.
Immutable reports	Edits construct validated replacements. This simplifies rollback and prevents silent changes to the service collection.
Two Item subtypes	Makes inheritance and polymorphic report types explicit. A single concrete class with a Type field would also be viable.
Atomic replacement + lock	Avoid overlapping app sessions and partial replacements on supported filesystems. Unsupported atomic moves produce an error.
Optional terminal styling	Restrained colour and borders aid readability. Plain mode supports evaluators and redirected output.

Simple rules make the score easy to inspect, but they also have limits. Location matching ignores case and extra spaces; it does not know that two different place names may refer to the same building. The 30-day window and 60-point threshold are design choices. Their usefulness would need to be measured against labelled campus reports.

SECTION 09

Implementation Details

The application contains ten source files in six packages. Main reads the command-line options, opens FileManager, creates ReportService, and enters the menu loop. Without a data option, it uses data/items.csv. Demo mode copies the fictional examples to a separate file on first use.

Report objects store their fields as private final values. Item.create constructs the appropriate lost or found subtype. ReportService assigns a UUID to each new report. An edit preserves the ID and type, while resolve and reopen actions change the status. Each operation prepares a replacement list, saves it, and only then updates the service.

Before scoring, Matcher excludes identical IDs, reports of the same type, resolved cases, and different categories. The found date must be between the lost date and 30 days later, inclusive. Keywords are taken from the title and description, converted to lowercase, and stored as sets. Repeated words therefore earn no extra points.

```
score = category + location + date + keywords

category = 35
location = same normalized location ? 25 : 0
date = round(25.0 * (30 - dayGap) / 30)
keywords = unionSize == 0 ? 0
: round(15.0 * sharedSize / unionSize)
```

A result is shown when its total reaches 60. Results are ordered by higher score, then shorter date gap, then candidate ID. Alongside the total, each Match record keeps category, location, date, and keyword points, the day gap, and the shared keywords.

Search applies all supplied filters together and includes both ends of a date range. FileManager checks the CSV before returning any records. To save, it writes a neighbouring temporary file, flushes it, and requests an atomic replacement. The file lock and channel are closed with try-with-resources. Invalid CSV stops loading without clearing the original file.

SECTION 10

Screenshots / Results

The following figures present actual terminal output from the supplied fictional dataset. They are formatted text captures rather than desktop screenshots. To reproduce the matching example, start with `--demo --plain`, choose option 3, and enter L001. Choose option 5 to view statistics. The images are embedded in this report; sample records are in `examples/items.csv`.

```
MATCHING / captured CLI output
Rendered from program stdout; not a desktop screenshot.

BLUE STEEL BOTTLE
LOST / OPEN
+-----+
ID: L001
Category: OTHER
Location: Central Library
Incident date: 2026-09-01
Description: Blue steel bottle with moon sticker
Contact / reference: Demo desk A
Similarity scores are suggestions, not proof of ownership.

+-----+
BLUE STEEL BOTTLE
FOUND / OPEN
+-----+
ID: F001
Category: OTHER
Location: Central Library
Incident date: 2026-09-02
Description: Blue steel bottle with moon sticker
Contact / reference: Demo desk B
SIMILARITY 99/100
Category 35/35 | Location 25/25 | Date 24/25 | Keywords 15/15
Found 1 day(s) after loss. Shared keywords: blue, bottle, moon,
steel, sticker

+-----+
BLUE BOTTLE
FOUND / OPEN
+-----+
ID: F002
Category: OTHER
Location: Cafeteria
Incident date: 2026-09-03
Description: Blue plastic bottle
Contact / reference: Demo desk C
SIMILARITY 63/100
Category 35/35 | Location 0/25 | Date 23/25 | Keywords 5/15
Found 2 day(s) after loss. Shared keywords: blue, bottle
```

Figure 7. L001 retrieves F001 with similarity 99/100. All four components are visible.

F001 receives 99 points: 35 for category, 25 for location, 24 for the one-day gap, and 15 for keywords. F002 receives 63 points: 35 for category, 0 for location, 23 for date proximity, and 5 for keywords. Both pass the threshold, although the first has stronger supporting details.

Report totals in the original demo

STATISTICS / captured CLI output

Rendered from program stdout; not a desktop screenshot.

CAMPUS SNAPSHOT

Counts describe reports, not verified item returns.

```
+-----+
Total reports: 7
LOST: 3
FOUND: 4
OPEN: 5
RESOLVED: 2
ELECTRONICS: 0
ID_CARD: 0
KEYS: 2
BOOKS: 0
CLOTHING: 0
BAGS: 2
OTHER: 3
```

Figure 8. Seven reports: three LOST, four FOUND, five OPEN, two RESOLVED.

The sample contains seven report rows: three lost and four found. Five are open and two are resolved. These are report counts, not a count of items returned to their owners. Contacts in the sample are fictional desk references, and no real campus dataset was used to measure matching accuracy.

SECTION 11

Testing Approach

All automated checks are in `test/ProjectTest.java`. The test scripts compile the application and this file before running the checks. A failed assertion stops the run with a nonzero exit code. Tests create temporary datasets, including for subprocesses that exercise the real terminal entry point.

Test group	Checks	Coverage
Matching	19	Eligibility, scoring, boundaries, token normalization, empty sets, tie ordering.
Input validation	14	Blank/long/control input, leap days, future dates, enums, normalization.
CSV storage	14	UTF-8 quoting, restart, malformed data, case-insensitive duplicate IDs, locks and rejected-save preservation.
Report operations	16	CRUD, filters, dates, status, counts, immutable snapshots, failed-save memory rollback.
CLI workflows	13	Actual create/edit/search/resolve/reopen/delete flows, cancellation, EOF, bad options and plain mode.
Synthetic benchmark	1	Correct result count for 10,000 eligible candidates.

The revised package was tested on 17 September 2026 using Microsoft OpenJDK 21.0.12 on Windows. All 77 checks passed. The 10,000-candidate matching run took 118.46 ms, below the local target of 2 seconds. Application and test code also compiled with `--release 21 -Xlint:all` without warnings. Run `test.cmd` on Windows or `sh test.sh` on macOS/Linux to print the individual results.

The benchmark measures matching for one report; it excludes file loading and saving. A single timing sample cannot predict performance on every computer. The tests check specified behaviours, not the accuracy of real-world ownership suggestions. macOS/Linux scripts are supplied but have not been run on those operating systems.

SECTION 12

Challenges Faced

CSV fields can contain commas and quotation marks. Splitting every row on commas would break those values. FileManager instead reads quoted fields character by character and doubles internal quotes when saving. The storage tests include commas, quotes, and accented characters.

A save can fail after a user has requested a change. ReportService handles this by keeping the old list until the replacement file is written successfully. One test deliberately places a nonempty directory at the CSV path to trigger a write failure and confirms that the service keeps its previous records.

Two reports may share a category and location while describing different items. The matcher therefore displays the separate score components and shared keywords. The operator still needs to compare the details and confirm ownership before returning an item.

Terminal rendering varies between environments. ConsoleUI keeps colour and border formatting in one place, and --plain produces ASCII output without ANSI sequences. Scripted input is supported so that complete menu workflows can be tested automatically.

ID comparison needed to be consistent throughout the application. An earlier review found that IDs such as L001 and l001 could be loaded together even though lookup treated them as the same ID. Loading, saving, and matching now follow the same case-insensitive rule. Three regression checks cover the correction.

SECTION 13

Learnings & Key Takeaways

Separating responsibilities makes a small program easier to understand. Models describe valid data, the service manages changes, storage handles files, and ConsoleUI handles interaction. The matching rules can be exercised without a terminal session or a storage dependency.

Sets fit the keyword comparison problem because duplicates should not change the score. The intersection records shared words, while the union records all distinct words across both reports. Their ratio gives a comparison that can be checked with a short example.

Persistence requires checking failure cases as well as successful saves. Reloading records, testing quoted text, and forcing a write failure reveal problems that a normal demonstration may miss. Updating memory only after saving keeps the application state consistent.

A validation rule should mean the same thing at every layer. Case-insensitive lookup must be paired with case-insensitive duplicate checks. Boundary tests for dates, thresholds, and empty keyword sets help make such rules explicit.

A numerical score is more useful when its meaning is clear. The application shows the evidence behind a suggestion and leaves the ownership decision to a person. Its local storage design also defines a practical limit: separate installations cannot collaborate without a different storage and access model.

SECTION 14

Future Enhancements

Campus location aliases could map common abbreviations to a standard name. This would help when students describe the same place differently, provided ambiguous aliases are tested carefully.

Matching quality could be measured using consented, anonymised report pairs with manually verified outcomes. Separate evaluation data would allow different weights and thresholds to be compared using precision, recall, and examples of incorrect suggestions.

A verified-return operation could link a lost report to a found report after ownership is checked. Both statuses could then be updated together with a resolution reference. The current application manages each report independently.

A shared service and database would be needed for access from several computers. That extension would also require authentication, permissions, privacy decisions, and safe handling of concurrent changes.

Other improvements include optional backups, an audit trail, student usability feedback, and testing on more operating systems. These are proposed extensions; they are not part of the present implementation.

SECTION 15

References

1. VITyarthi. Build Your Own Project: General Project Instructions & Submission Guidelines.
<https://vityarthi.b-cdn.net/VITyarthiProjects/BuildYourOwnProjectVITyarthi.pdf>
2. VITyarthi. Course portal project-submission instructions: repository visibility, repository URL format, terminal execution, report submission, originality, and evaluation policy.
3. Oracle. Java Platform, Standard Edition 21: The javac Command.
<https://docs.oracle.com/en/java/javase/21/docs/specs/man/javac.html>
4. Oracle. Java SE 21 API Specification: LocalDate.
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/LocalDate.html>
5. Oracle. Java SE 21 API Specification: Files.
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/file/Files.html>
6. Project materials: application source under src/com/lostfound; automated checks in test/ProjectTest.java; fictional sample records in examples/items.csv; setup and testing instructions in README.md.